

Task Day09

1. Vodafone Intelligent Solutions (_VOIS)

Why: It's one of the largest technology hubs in Egypt and globally, serving as Vodafone's internal software development powerhouse. _VOIS handles massive-scale telecom systems, digital transformation, and enterprise solutions. It's a benchmark for how Egyptian talent can serve Fortune 500 companies with complex, high-availability systems.

2. ITWorx

Why: A pioneer in the Egyptian software industry since 1994, ITWorx has built a strong reputation in custom software development, digital transformation, and Microsoft-centric solutions. They've worked with governments, enterprises, and international clients, making them a symbol of Egypt's capability to deliver enterprise-grade software for over two decades.

3. Sprints.ai

Why: Represents the new wave of Egyptian tech—focused on AI-driven education and talent development. They combine a software house model with an EdTech mission, building platforms that upskill developers while delivering software solutions. This dual approach addresses Egypt's core challenge: bridging the gap between academic output and industry-ready engineering talent.

Report

Service Layer: Concept & Implementation Report

1. What is the Service Layer?

The Service Layer sits between the Controller (Presentation) and the Repository (Data Access). It contains business logic – the rules, calculations, validations, and workflows that define *how* your application behaves.

2-Why do we need it?

Without Service Layer	With Service Layer
Business logic scattered in Controllers	Centralized, reusable business logic
Controllers become fat and hard to test	Controllers stay thin (Single Responsibility)
Code duplication across controllers	Services can be reused by multiple controllers
Hard to unit test business rules	Easy to mock and test services in isolation
Repository leaks into presentation	Clean separation of concerns

3-Key Responsibilities

Responsibility	Example
Business Rules	"A product price cannot be negative"
Data Validation	"Email must be unique across the system"
Orchestration	"Create order → Deduct stock → Send email"
Mapping	Convert Product entity → ProductDto
Transaction Management	Save multiple related entities atomically
External Service Calls	Call payment gateway, send SMS, etc.

4-Implementation in [ASP.NET](#) Core

Step 1: Define the Service Interface

```
// Services/IProductService.cs
public interface IProductService
{
    Task<IEnumerable<ProductDto>> GetAllAsync();
    Task<ProductDto?> GetByIdAsync(int id);
    Task<ProductDto> CreateAsync(CreateProductDto dto);
    Task<ProductDto?> UpdateAsync(int id, UpdateProductDto dto);
    Task<bool> DeleteAsync(int id);
    Task<bool> ApplyDiscountAsync(int productId, decimal percentage);
}
```

Step 2: Implement the Service

```
// Services/ProductService.cs
public class ProductService : IProductService
{
    private readonly IProductRepository _repository;
    private readonly IEmailService _emailService;
    private readonly IMapper _mapper;

    public ProductService(
        IProductRepository repository,
        IEmailService emailService,
        IMapper mapper)
    {
        _repository = repository;
        _emailService = emailService;
        _mapper = mapper;
    }

    public async Task<IEnumerable<ProductDto>> GetAllAsync()
    {
        var products = await _repository.GetAllAsync();
        return _mapper.Map<IEnumerable<ProductDto>>(products);
    }

    public async Task<ProductDto?> GetByIdAsync(int id)
    {
        var product = await _repository.GetByIdAsync(id);
        return product == null ? null : _mapper.Map<ProductDto>(product);
    }
}
```

```

public async Task<ProductDto> CreateAsync(CreateProductDto dto)
{
    // Business Rule: Price must be positive
    if (dto.Price <= 0)
        throw new ArgumentException("Price must be greater than zero");

    // Business Rule: Name must be unique
    var existing = await _repository.GetByNameAsync(dto.Name);
    if (existing != null)
        throw new InvalidOperationException("Product name already exists");

    var product = _mapper.Map<Product>(dto);
    await _repository.AddAsync(product);
    await _repository.SaveChangesAsync();

    // Business Action: Notify admin
    await _emailService.SendAsync("admin@company.com", "New Product Added",
dto.Name);

    return _mapper.Map<ProductDto>(product);
}

public async Task<bool> ApplyDiscountAsync(int productId, decimal percentage)
{
    // Business Rule: Discount cannot exceed 50%
    if (percentage > 50)
        throw new ArgumentException("Discount cannot exceed 50%");

    var product = await _repository.GetByIdAsync(productId);
    if (product == null) return false;

    product.Price -= product.Price * (percentage / 100);
    await _repository.SaveChangesAsync();
    return true;
}
// ... other methods
}

```

Step 3: Register in DI Container

```
// Program.cs
builder.Services.AddScoped<IProductRepository, ProductRepository>();
builder.Services.AddScoped<IProductService, ProductService>();
builder.Services.AddScoped<IEmailService, EmailService>();
```

Step 4: Use in Controller

```
// Controllers/ProductController.cs
public class ProductController : Controller
{
    private readonly IProductService _productService;

    public ProductController(IProductService productService)
    {
        _productService = productService;
    }

    public async Task<ActionResult> Index()
    {
        var products = await _productService.GetAllAsync();
        return View(products);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<ActionResult> Add(CreateProductDto dto)
    {
        if (!ModelState.IsValid) return View(dto);

        try
        {
            var product = await _productService.CreateAsync(dto);
            return RedirectToAction("Index");
        }
        catch (Exception ex)
        {
            ModelState.AddModelError("", ex.Message);
            return View(dto);
        }
    }
}
```